

Solver Competence Before Executable Memory: A Witness-CL Development Report

Samuel Mausberg

AI-assisted research draft; independent replication pending

Abstract

The previous Witness-CL pilot failed to establish solver competence: all six memory methods scored zero on eight warm questions. We address this prerequisite with seven recorded development diagnostics, ordinary-SQL interface repairs, and three separately frozen follow-up attempts. A larger model with explicit live interaction and thinking reaches 7/8 in the diagnostic. In the final fresh development block, full history, evolving text and checked executable memory score 8/8, 6/8 and 7/8 respectively. The predeclared all-method competence gate therefore stops evaluation. Seven program proposals yield two finite reconstruction admissions, but no program reuse or composition. An offline intervention finds one admission’s wrapper completely independent of its learned relation. We implement a bounded, feedback-guided repair prototype with a paid relation-dependence check and test it offline. The experiments demonstrate improved solver competence and identify repairable interface and discovery failures; they do not establish the intended conjunction of reusable abstraction discovery, transfer, interaction savings and retention.

1 Research question

The target remains an online agent that discovers executable abstractions from its own experience, applies them on genuinely new compositions without task identifiers or an enumerated semantic feature library, reduces interaction against strong full-history and evolving-memory baselines, and preserves earlier behavior. A useful empirical test must satisfy these prerequisites together. Correct ordinary SQL alone is not executable-memory transfer. Smaller prompts are not automatically fewer interactions. Compiler identities do not establish learned behavior.

The v8 pilot could not test this claim because every method failed its initial competence check. This report preserves that negative result and investigates the bottleneck. All new model runs are development experiments. Settings were adapted on seed 92000, then each follow-up configuration was frozen before its fresh seed. We make no heldout claim, pool no adaptive accuracies, and estimate no population confidence interval from one correlated stream.

2 Common task and interface

The environment supplies a scalar question, opaque SQLite schemas and a queryable documentation catalog. The catalog explains monetary units, missing values, row multiplicity and current-profile conventions. The agent receives no numerical target, task family identifier or enumerated semantic feature library. SQL syntax, the pretrained model and the action interface remain inductive biases.

Each episode permits eight SELECT attempts, including failed SQL, checks and post-answer reconstruction, and ten solve-action calls. Results have fixed row, byte and execution bounds. The executor permits read-only SQL over fixed private tables; it denies writes, external databases, extensions and disallowed functions or virtual modules. Correctness is one binary outcome after the answer. Ordinary learning may consume its remaining query allowance after that outcome, without revising the score. Later evaluation panels freeze memory.

Two implementation defects were repaired in new v9 modules. First, ordinary SQL had inherited restrictions intended for stored fragments; the revised action parser allows valid terminal semicolons, comments and unused parameter bindings, while SQLite still validates execution. Second, the old authorizer denied SQLite’s anonymous reads of derived relations. The revised rule admits these reads within the fixed connection while continuing to deny registered virtual modules and internal tables. An otherwise correct diagnostic current-profile CTE triggered the old denial. Its original score remains unchanged.

3 Solver and runtime repairs

Table 1 records every solver diagnostic. Four Qwen3-4B Q8_0 configurations, including explicit evidence instructions and sampled/thinking decoding, score 0/8. Qwen3.5-9B Q8_0 scores 1/8 with the evidence prompt and 0/8 with a live interaction prompt under nonthinking decoding. Live interaction with thinking reaches 7/8. This is a successful competence screen for one configuration, not an isolated estimate of model-size or reasoning effects.

The 9B model uses pinned Unsloth GGUF weights of the official Qwen3.5-9B model; provenance records

Table 1: All adaptive solver diagnostics on development seed 92000.

Setting	Correct	Calls	Tokens
4B original	0/8	16	10,438
4B evidence	0/8	17	13,985
4B sampled	0/8	18	14,957
4B thinking	0/8	14	25,706
9B evidence	1/8	23	23,588
9B live	0/8	20	25,799
9B live + thinking	7/8	28	46,959

Table 2: Final fresh development warm block. Accuracy and costs are shown separately; these are not matched-accuracy efficiency results.

Method	Correct	SELECTs	Calls	Tokens
Full history	8/8	9	17	50,178
Evolving text	6/8	14	30	55,808
Checked programs	7/8	22	31	49,700

the repository revision, expected and actual SHA-256, runtime revision and effective generation settings. The official model and quantization sources are available at <https://huggingface.co/Qwen/Qwen3.5-9B> and <https://huggingface.co/unsloth/Qwen3.5-9B-GGUF>. We did not alter model weights or supply evaluator solutions. Inference used an authenticated loopback server with no model authority over shell or network tools.

Three subsequent attempts used fresh development seeds 92001, 92002 and 92003. The first stopped after two records when the native JSON-schema grammar rejected a 4096-character string bound during reflection. The second used JSON-object mode, encountered malformed memory updates and unfinished reasoning at the forced cutoff, and was intentionally stopped after seven records. Its pending request timed out during shutdown. Both failed invocations and their unavailable usage remain part of the record.

The final, separately frozen adapter removes only unsupported large string bounds from the native wire schema. Original host-side shape, type and length validation remains active. It disables the native forced reasoning cutoff, retains the 2048-token solving ceiling, and gives all memory reflection/proposal calls a common nonthinking 4096-token ceiling. Temperature is 0.6, top-p 0.95, top-k 20, min-p 0, presence penalty 1.5 and seed 42. Native and client context ceilings are 65,536 tokens. The adapter neither repairs model responses nor rewrites SQL. Solving and memory-phase settings are recorded per call.

4 Fresh competence comparison

The final attempt starts all three memories empty on seed 92003. Full history retains every legal ordinary transcript without truncation. Evolving text can retain 16 entries of up to 4096 characters under a 64 KiB active cap, including literal SQL. Checked executable memory may propose a new parameterized SELECT relation and reconstruct its own successful scalar result through an outer query. No control is given hidden catalog contents, gold queries or task IDs.

Table 2 shows full history at 8/8, evolving text at 6/8 and checked programs at 7/8. The predeclared gate requires at least 7/8 from every method. Consequently the run stops after 24 of the planned 144 records. No old-task, post-warm or final composition panel executes. Evolving text remains an unqualified control; weaknesses of this implementation do not establish superiority over the best possible text-memory baseline.

The text-memory errors have concrete warning signs: an average omits the documented NULL-as-zero convention, and a later regional gross query uses the wrong join key and raw unit amounts. The retained summaries had dropped the relevant earlier details. These observations motivate preserving exact observed documentation and successful SQL, without proving a single cause of failure.

The final block consumes 45 SELECT attempts, 78 model calls and 155,686 tokens in 401.03 seconds. All final-block usage is known. Costs include reflection and failed reconstruction. The accuracy differences make this unsuitable as a matched-accuracy efficiency comparison. In particular, the 7/8 program-arm score comes entirely from ordinary SQL: there are zero USE or COMPOSE actions and zero applicability checks.

5 Program discovery audit

Seven proposal calls produce six paid reconstructions and two finite admissions. Five proposals fail because of incorrect outer relation scope, an out-of-range guard index, a text binding containing literal quotes, or an equality predicate bound to NULL. The model repeatedly names a private inner alias where the host exposes only the CTE `reused`. These are actionable failures in the learning interface; they are not successful abstractions.

One admitted relation counts distinct refunded orders and the wrapper sums that scalar. Its integer parameter is an all-or-nothing truth gate. Emptying the relation changes the wrapper result from 22 to NULL; replacing it with a sentinel changes the result accordingly. This establishes one-instance dependence of the wrapper, not a useful parameterized row representation. Moreover, this target is structurally 22 in the generator, so it is unsuitable as sole transfer evidence.

The other admitted relation computes a Gold-tier scalar total, but its wrapper repeats the physical-table

SQL and never reads the learned relation. Deleting, emptying, replacing or changing the inner binding leaves 1350.25 unchanged. The stored SQL independently produces its training answer, but the paid reconstruction witness did not establish that its output contributed. This is a concrete admission weakness, not evidence of discovered reuse.

These interventions are 11 explicitly separate auditor SQL executions on saved development fixtures. They add no model calls, learner observations or memory updates and change no recorded score. They are neither changed-binding behavior by the agent nor a fresh transfer experiment.

6 Bounded repair prototype

An unintegrated prototype permits at most three feedback-guided proposals after a correct ordinary episode. It gives actual compiler or reconstruction errors to the model while preserving the original source and guard prefix. Every executed check consumes remaining episode SELECTs. Failed checks can inform repair but cannot become new source witnesses. Learning is disabled on panels.

After successful reconstruction, admission is staged in a separate memory copy. An additional paid probe empties the learned relation while preserving its output columns. Admission commits only if this complete probe changes the outer result. A wrapper that ignores the relation is therefore rejected on the observed fixture. Offline tests exercise failed-alias repair, budgets, fixed provenance, unused relations and panel immutability. No actual-model experiment has run this prototype, so it contributes no empirical successes above.

The intervention is conservative: a useful relation may have the same result when empty on a particular instance and be rejected. Conversely, passing does not establish semantically meaningful holes, fresh composition or retention. A constant scalar with a dependent wrapper can still pass. Those properties require additional, separately specified evidence.

7 Accounting and next step

Across all attempts, 89 episode records contain 244 generation requests and 366,795 known tokens. Two failed calls have unknown usage. Their declared input/output reservations total 9109 tokens, giving a protocol-based upper bound of 375,904, not an exact measured total. Model-experiment time is 1104.03 seconds (18.40 minutes), excluding separately recorded download and startup. This remains within the original 45-minute, 4M-token and 1600-request ceilings. The owned server was stopped and its temporary key removed.

Saved-data audits check frozen sources, exact prompts, query outcomes, memory transitions, charged checks, known/unknown usage and stopping rules. They pass for all three stream attempts and all diagnostics. Their guarantees concern consistency of saved evidence, not cryptographic authentication of inference, weights, tokenization or clocks. The earlier Lean compiler contracts are unchanged; no new theorem substitutes for a behavioral result.

The next study should first qualify an evolving-memory control that preserves observed documentation and successful SQL, then freeze and evaluate the repair prototype. A convincing mechanism trace needs a useful learned relation, meaningful rebinding, a new outer operation and correct answers dependent on the retrieved program. Old/new panels and complete cost accounting must follow on multiple disjoint streams before any population claim.

There is also an objective-design constraint: once full history can answer with one SELECT, this checked-use path already spends at least two, one for its guard and one for execution, before amortized synthesis. SQL interaction savings need enough baseline exploration to offset this overhead. Token and latency savings would be different endpoints. The present contribution is a competent solver configuration, identified interface failures and a testable repair, with the original research claim still open.